The Journal of Open Source Software

CWIPI: Coupling With Interpolation Parallel Interface

Eric Quémerais [1], **Bastien Andrieu** [1], **and Karmijn Hoogveld** [1]

1 DMPE, ONERA, Université Paris-Saclay, 92320 Châtillon, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @KennethKwabenaKenney
- @GuilloteauQ
- @MakisH

Submitted: 03 April 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

CWIPI is an open-source library dedicated to the parallel coupling of independent simulation codes for multi-physics research, a field also covered by tools such as MUI (Tang et al., 2015) and preCICE (Chourdakis et al., 2022). It is aimed at anyone wishing to conduct studies and research in this field. It provides MPI-based communication and fully distributed data exchange on non-coincident meshes, including on-the-fly spatial interpolation. CWIPI stands out for its support of a wide variety of spatial interpolation families and a broad range of geometric element types. This extensive spatial capability allows users to leverage solver-specific numerical methods, including finite volumes, finite elements, discontinuous Galerkin, and spectral methods. Advanced users can further combine CWIPI's geometric algorithms with solvers' internal interpolations through callbacks, improving accuracy across highly heterogeneous solvers.

CWIPI supports a wide range of interface geometries, from linear, surface, and volumetric elements to higher-order elements with curved edges and faces. Performance-critical algorithms have been specifically optimized for handling coupling cases involving dynamic or adaptive meshes. Recent developments in the ParaDiGM library (Quémerais et al., 2026) now provide dynamic load and memory balancing for large-scale simulations. Ongoing work includes GPU acceleration of geometric algorithms (Cazalbou et al., 2024) and the development of temporal interpolation methods (Simon et al., 2025).

Statement of Need

Multi-physics problems can be approached either through monolithic methods — where all physical phenomena are solved simultaneously within a single solver — or through partitioned methods, where independent solvers exchange data across shared interfaces. Partitioned approaches are particularly attractive in high-performance computing (HPC) contexts, as they allow the reuse of existing specialized codes while maintaining their native parallel architecture.

Robust and efficient coupling tools are essential to enable data exchange between solvers operating on distinct meshes, geometries, and discretization schemes. CWIPI was initially developed in 2009 to provide an open-source solution complementary to MPCCI (Joppich & Kürschner, 2006), whose closed-source nature limited flexibility and transparency for multi-physics research.

To ensure a straightforward and efficient deployment of coupled simulations, an HPC coupling tool must abstract the complexity of parallelism by interacting solely with data local to each MPI rank. Furthermore, it must deliver high-performance spatial interpolation algorithms whose execution time remains negligible compared to a single solver iteration, thereby avoiding any penalty on the overall simulation wall-clock time. In an HPC context, the most expensive and complex part of these algorithms lies in the underlying parallel geometric computations, particularly for 3D volumetric couplings with moving meshes; this is the challenge CWIPI was primarily designed to address (Andrieu et al., 2026). Alongside these, it must provide

DRAFT

temporal coupling schemes—or at least the essential building blocks to implement them. While meeting these requirements is sufficient for standard use cases, the ability to implement specific spatial interpolation methods is critical to achieve a global accuracy order close to the solvers' internal precision. This necessity was demonstrated by (Léger et al., 2012) for an isophysical configuration based on the linearized Euler equations: by exposing CWIPI's geometric algorithms to the solvers' internal interpolation capabilities via callbacks, and by employing the same Runge-Kutta scheme with coupling performed at the sub-iteration level, a global accuracy order of 3.5 was preserved when coupling a 4th-order finite difference code with a 4th-order discontinuous Galerkin solver.

State of the Field

Existing coupling frameworks differ significantly in scope, geometry support, and interpolation capabilities. MUI (Tang et al., 2015) primarily supports point clouds and focuses on general-purpose temporal and spatial interpolation. preCICE (Chourdakis et al., 2022) supports point clouds or meshes composed of planar triangular/quadrilateral elements, or tetrahedral cells, provides a broad ecosystem of adapters for established simulation codes, and integrates numerically black-box temporal coupling algorithms. OpenPALM (Duchaine et al., 2015) is a dynamic code coupler that uses a supervision process that orchestrates temporal coupling algorithms, which are defined beforehand by the user through the PrePALM graphical user interface. In this architecture, OpenPALM operates as a higher-level coupler, relying entirely on CWIPI as its underlying spatial interpolation and low-level communication engine. Although a comparative study of coupling tools within an HPC framework (Rubin, 2022) has been conducted, it is not exhaustive; it mentions other tools besides those cited above but only discusses CWIPI through its utilization via OpenPALM.

Regarding temporal coupling algorithms, CWIPI provides the building blocks to define the orchestration of exchanges. The user defines the temporal coupling algorithm in the solver source code, in a decentralized manner. Such algorithms are typically relaxation methods (such as Aitken, implemented in MUI and preCICE) or quasi-Newton schemes (implemented in preCICE).

Regarding spatial interpolation of fields, a concept designated as "mapping" or "sampling" depending on the framework, CWIPI implements spatial interpolation methods based on local neighboring data, an implementation well-suited for HPC (Andrieu et al., 2026). Other coupling tools for HPC also use such interpolation methods: MUI (Liu et al., 2026) and preCICE (Schneider & Uekermann, 2025) offer local RBF-type interpolation methods using the Partition-of-Unity approach. In addition to that, MUI includes gaussian sampler (e.g., for molecular dynamics applications) (Tang et al., 2015) and preCICE includes linear interpolation methods among neighboring points and global (dense or sparse) RBF-type methods (Chourdakis et al., 2022; Schneider et al., 2023).

CWIPI offers a rich geometric representation of interfaces, including linear, surface, and volumetric elements (similarly to other tools), as well as polygons, polyhedra, and high-order elements with curved edges and faces, which offer additional versatility compared to other tools. To exploit these geometries, CWIPI provides three families of interpolation methods based on an underlying geometric algorithm: one based on point clouds, one based on locating target points within the source mesh, and one based on the intersection of source and target meshes. For each of these, a default method is provided, supplemented by a callback interface allowing users to implement the interpolation method best suited to their specific context. If MUI also allows users to write a customized callback, this is limited to point clouds; conversely, CWIPI extends this capability to the three geometric families. Within a CWIPI callback, parallelism management is entirely transparent: the user works exclusively with local data, without having to manage remote communications, except in the specific case where they invoke internal functions of their own code requiring their intracommunicator. It should be noted that the

DRAFT

family based on locating target points was the first to be implemented, which explains its privileged use in the vast majority of CWIPI applications.

Strategies for launching and communication differ significantly across frameworks. MUI and CWIPI follows an approach based on MPI MPMD (Multiple Program, Multiple Data), whereas preCICE follows a decoupled approach where each code is launched via its own independent MPI command (MPI Port), with the connection between processes established a posteriori. CWIPI also offers a TCP/IP communication mode, which is rarely used in HPC simulations. This mode was developed to couple CWIPI with commercial simulation codes that embed their own MPI implementation in their binaries.

CWIPI offers high flexibility in distributing codes across MPI ranks: a code can be placed on any rank, and several codes can share the same ranks (see the `c_new_api.c` test).

Consequently, the intersection of the MPI intra-communicators of different codes is not necessarily empty. In the limiting case where codes share all their ranks, their intra-communicators become duplications of the same communicator. This architecture offers high flexibility: the same coupling case can be handled with disjoint intra-communicators or fully overlapping ones, without any assumptions regarding the location of the coupling interface. In practice, although this capability is available in C and Fortran, it will mainly be used in Python, since the flexibility of this language makes it easy to load the module for a given coupled code depending on the MPI rank number, and thus to distribute the codes as desired. This capability therefore makes it possible to adjust this distribution in order to improve load balancing across the whole coupled simulation. CWIPI does not provide a capability for searching for an optimal distribution, but this could constitute a future research direction.

Software Design

Since version 1.0, CWIPI is built on top of the ParaDiGM library (Quémerais et al., 2026), originally created specifically to meet the needs of CWIPI. ParaDiGM provides the low-level geometric algorithms and parallel data structures. More broadly, ParaDiGM was designed to mutualize parallel geometric algorithms that share common requirements across other contexts, such as dynamic mesh adaptation and, more generally, any geometric co-processing tasks performed by solvers during runtime.

CWIPI provides three APIs C (which can be used from C++ codes as well), Fortran, and Python/NumPy — making it compatible with a wide range of scientific codes. The integration of CWIPI into an existing solver is designed to be minimally intrusive: it does not require modifying the code structure, and the API relies solely on simple arrays, keeping the coupling layer lightweight and straightforward to adopt (see the Quickstart for a minimal example).

To support various discretization schemes, CWIPI's three interpolation families provide distinct default approaches and data structures for customized implementations. The first family, based on searching for the k-nearest source nodes for each target point, operates on nodes, cell centers, or user-defined points for both source and target locations. Its default method relies on a least squares approach, and its callback interface exposes, for each target point, the local data containing the k-nearest source points, their distances, and the associated source field values. The second family focuses on the location of target points within the source mesh. It maps source fields defined at nodes or cell centers onto target fields located at nodes, cell centers, or user-defined points. By default, it uses the Lagrange basis functions of the source elements; for custom interpolations, the callback interface provides the data local to each source element, specifically the target points it contains (or their projections) and the source element fields. The third family is based on the intersection of source and target meshes, where both fields are strictly located at cell centers. Its default method performs a weighting by the volumes of the intersecting polyhedra, and the callback interface supplies each target element with its exact intersection volumes and the fields defined on the intersecting source polyhedra.

DRAFT

At each coupling step, the execution sequence typically proceeds as follows: first, the geometric algorithm corresponding to the selected interpolation family is invoked; second, the physical fields are exchanged across the communication graph established by the preceding geometric step; and finally, the fields are interpolated onto the target mesh. However, if the chosen family is based on the location of target points within the source mesh, the field interpolation is performed directly by the source code, which inverts the order of the last two steps. Furthermore, in cases where the mesh interface remains fixed throughout the simulation, the geometric algorithm is invoked only once during the initialization phase, and its results are subsequently stored in memory for all remaining coupling steps.

Within this coupling communicator, the geometric algorithms linked to the interpolation families generate optimized point-to-point communication graphs between the processes of the coupled codes. Each geometric algorithm produces its own communication graph. This graph also depends on the location of the fields' degrees of freedom (nodes, cell centers, etc.). Since the location of the degrees of freedom and the choice of interpolation family are properties specific to each field, CWIPI is capable of managing several distinct communication graphs between the codes for a single coupling instance.

A major design challenge lies in the **performance of geometric algorithms**, especially when the coupling interface moves during the simulation or when solvers employ dynamic mesh adaptation. In the 0.x branch of CWIPI, algorithms relied on the FVM library from Code_Saturne (Archambeau et al., 2004), which later evolved into PLE, a lightweight coupling library (Fournier, 2020). New algorithms (Andrieu et al., 2026) developed within ParaDiGM provide **dynamic load and memory balancing** at each coupling step, which is critical because coupling problems are inherently unbalanced across MPI processes. These developments bring significant performance gains for large-scale and complex simulations.

Future Directions

Two main research directions are currently pursued. The first concerns the **GPU porting** of the most computationally expensive geometric algorithms, for which new algorithms have recently been designed (Cazalbou et al., 2024) and will be integrated in an upcoming release. The second focuses on the development of **temporal interpolation methods** (François & Massot, 2023; Simon et al., 2025), complementary to those provided by existing frameworks such as preCICE, aiming to improve accuracy and flexibility for dynamic interfaces and heterogeneous solvers.

Research Impact Statement

From a scientific perspective, CWIPI is primarily used within the aeronautics and aerospace community from which it originated. The library is integrated into several codes developed at ONERA, including elsA (Cambier et al., 2013) for computational fluid dynamics, CEDRE (Refloch et al., 2011) for multiphysics simulations, Z-set (Garaud et al., 2019) for structural and materials mechanics, MoDeTheC (Dellinger et al., 2024) for thermal degradation of materials, and SPACE (Delorme et al., 2005) for acoustic propagation. Through these tools, CWIPI enables advanced multiphysics simulations in aerothermodynamics (Errera et al., 2019), aeroacoustics (Langenais et al., 2019; Moratilla-Vega et al., 2022), magnetohydrodynamics (Rocamora et al., 2025), electric arc propagation, and fire safety (Dellinger et al., 2023).

CWIPI is also employed as both an internal and external coupling library in leading combustion simulation environments such as YALES2 (Moureau et al., 2011) and AVBP (Schonfeld & Rudgyard, 1999). A notable HPC application (Dombard et al., 2018) combining CWIPI with AVBP on a complex geometry demonstrated the robustness and scalability of the coupling approach on massively parallel architectures. Its capabilities were highlighted as early as 2017 (Duchaine et al., 2017) and confirmed by an invitation to the ExCALIBUR workshop (Andrieu & Quémerais, 2021). Recent work in aeroelasticity with YALES2 (Fabbri et al., 2023) further

DRAFT

demonstrates its versatility. CWIPI's integration into the software stack of national HPC facilities such as the CCRT confirms its recognition at a national level.

Beyond its original ecosystem, the open-source community has adopted CWIPI through GitHub-based interfacing projects with OpenFOAM (Moratilla, 2022; nd-92, 2025; Weller et al., 1998) and Nektar++ (Cantwell et al., 2015), enabling applications in aeroacoustics and data assimilation (Valero, 2026). Finally, CWIPI serves as the interpolation engine of the OpenPALM coupler (Duchaine et al., 2015), illustrating its central role in the broader multiphysics and HPC coupling ecosystem.

AI Usage Disclosure

Generative AI tools were used to assist with the writing of this paper: translation, formatting, and improving the fluency of the text. No AI was used in the development of the CWIPI software. All scientific content was written and validated by the authors.

Acknowledgements

The authors would like to thank the following people for their contributions to the development, testing, and dissemination of the CWIPI library within the community:

Romain Casta[1], Nicolas Dellinger[2], Florent Duchaine[1], Bastien Frisulli[2], Jean-Didier Garaud[2], Thomas Hennion[2], Stéphanie Lala[2], Xavier Lamboley[12], Bruno Maugars[2], Thierry Morel[1], Christophe Peyret[2]

[1] CERFACS — [2] ONERA

The authors also wish to thank BPI France, the Directorate General for Civil Aviation (DGAC), and the General Scientific Directorate of ONERA for their financial support.

Author Contributions

The contributions to this software are listed according to the CRediT taxonomy:

- **Eric Quémerais**: Conceptualization, Methodology, Software, Validation, Writing – Original Review & Editing, Project Administration, Funding Acquisition, Supervision
- **Bastien Andrieu**: Software, Validation, Writing – Original Draft
- **Karmijn Hoogveld**: Software, Validation, Writing – Original Draft

References

Andrieu, B., Maugars, B., & Quémerais, E. (2026). Dynamic load-balanced point location algorithm for data mapping. *Comptes Rendus. Mécanique*, *354*(G1), 53–70. https://doi.org/10.5802/crmeca.335

Andrieu, B., & Quémerais, E. (2021, December). The CWIPI library. *ExCALIBUR ISE Code Coupling Workshop 1*. https://excalibur-coupling.github.io/workshop1_content/slides/B-Andrieu_ISE_CWIPI.pdf

Archambeau, F., Méchitoua, N., & Sakiz, M. (2004). Code_Saturne: A finite volume code for the computation of turbulent incompressible flows. *International Journal on Finite Volumes*. https://hal.archives-ouvertes.fr/hal-01115371/document

Cambier, L., Heib, S., & Plot, S. (2013). The onera elsA CFD software: Input from research and feedback from industry. *Mechanics & Industry*, *14*, 159–174. https://doi.org/10.1051/meca/2013056

DRAFT

Cantwell, C. D., Moxey, D., Comerford, A., Bolis, A., Rocco, G., Mengaldo, G., De Grazia, D., Yakovlev, S., Lombard, J.-E., Ekelschot, D., Jordi, B., Xu, H., Mohamied, Y., Eskilsson, C., Nelson, B., Vos, P., Biotto, C., Kirby, R. M., & Sherwin, S. J. (2015). Nektar++: An open-source spectral/hp element framework. *Computer Physics Communications*, *192*, 205–219. https://doi.org/10.1016/j.cpc.2015.02.008

Cazalbou, R., Duchaine, F., Quémerais, E., Andrieu, B., Staffelbach, G., & Maugars, B. (2024). *Hybrid multi-GPU distributed octrees construction for massively parallel code coupling applications*. https://doi.org/10.1145/3659914.3659928

Chourdakis, G., Davis, K., Rodenberg, B., Schulte, M., Simonis, F., Uekermann, B., Abrams, G., Bungartz, H., Cheung Yau, L., Desai, I., Eder, K., Hertrich, R., Lindner, F., Rusch, A., Sashko, D., Schneider, D., Totounferoush, A., Volland, D., Vollmer, P., & Koseomur, O. (2022). preCICE v2: A sustainable and user-friendly coupling library. *Open Research Europe*, *2*, 51. https://doi.org/10.12688/openreseurope.14445.2

Dellinger, N., Donjat, D., Laroche, E., & Reulet, P. (2023). Experimental and numerical modelling of the interaction between a turbulent premixed propane/air flame and a composite flat plate. *Fire Safety Journal*, *141*, 103959. https://doi.org/10.1016/j.firesaf.2023.103959

Dellinger, N., Leplat, G., Huchette, C., Biasi, V., & Feyel, F. (2024). Numerical modeling and experimental validation of heat and mass transfer within decomposing carbon fibers/epoxy resin composite laminates. *International Journal of Thermal Sciences*, *201*, 109040. https://doi.org/10.1016/j.ijthermalsci.2024.109040

Delorme, Ph., Mazet, P.-A., Peyret, C., & Ventribout, Y. (2005). Computational aeroacoustics applications based on a discontinuous galerkin method. *Comptes Rendus Mécanique*, *333*(9), 676–682. https://doi.org/10.1016/j.crme.2005.07.007

Dombard, J., Duchaine, F., Gicquel, L. Y. M., Staffelbach, G., Buffaz, N., & Trébinjac, I. (2018). *Large eddy simulations in a transonic centrifugal compressor*. V02BT44A030. https://doi.org/10.1115/GT2018-77023

Duchaine, F., Berger, S., Staffelbach, G., & Gicquel, L. (2017). Partitioned high performance code coupling applied to CFD. In E. Di Napoli, M. A. Hermanns, H. Iliev, A. Lintermann, & A. Peyser (Eds.), *High-performance scientific computing* (Vol. 10164). Springer. https://doi.org/10.1007/978-3-319-53862-4_1

Duchaine, F., Jauré, S., Poitou, D., Quémerais, E., Staffelbach, G., Morel, T., & Gicquel, L. (2015). Analysis of high performance conjugate heat transfer with the OpenPALM coupler. *Computational Science & Discovery*, *8*(1), 015003. https://doi.org/10.1088/1749-4699/8/1/015003

Errera, M.-P., Moretti, R., Salem, R., Bachelier, Y., Arrivé, T., & Nguyen, M. (2019). A single stable scheme for steady conjugate heat transfer problems. *Journal of Computational Physics*, *394*, 491–502. https://doi.org/10.1016/j.jcp.2019.05.036

Fabbri, T., Balarac, G., Moureau, V., & Benard, P. (2023). Design of a high fidelity fluid-structure interaction solver using LES on unstructured grid. *Computers & Fluids*, *265*, 105963. https://doi.org/10.1016/j.compfluid.2023.105963

Fournier, Y. (2020). Massively parallel location and exchange tools for unstructured meshes. *International Journal of Computational Fluid Dynamics*, *34*(7-8), 1–20. https://doi.org/10.1080/10618562.2020.1810676

François, L., & Massot, M. (2023). Multistep interface coupling for high-order adaptive black-box multiphysics simulations. *Proceedings of the 10th International Conference on Computational Methods for Coupled Problems in Science and Engineering*. https://doi.org/10.23967/c.coupled.2023.027

DRAFT

Garaud, JD., Rannou, J., Bovet, C., Feld-Payet, S., Chiaruttini, V., Marchand, B., Lacourt, L., Yastrebov, V. A., Osipov, N., & Quilici, S. (2019, May). Z-set -suite logicielle pour la simulation des matériaux et structures. *14ème Colloque National en Calcul de Structures (CSMA 2019)*. https://hal.science/hal-02115875

Joppich, W., & Kürschner, M. (2006). MpCCI – a tool for the simulation of coupled applications. *Concurrency and Computation: Practice and Experience*, *18*(2), 183–192. https://doi.org/10.1002/cpe.913

Langenais, A., Vuillot, F., Troyes, J., & Bailly, C. (2019). Accurate simulation of the noise generated by a hot supersonic jet including turbulence tripping and nonlinear acoustic propagation. *Physics of Fluids*, *31*(1), 016105. https://doi.org/10.1063/1.5050905

Léger, R., Peyret, C., & Piperno, S. (2012). Coupled discontinuous galerkin/finite difference solver on hybrid meshes for computational aeroacoustics. *AIAA Journal*, *50*, 338–349. https://doi.org/10.2514/1.J051110

Liu, W., Mahfoze, O. A., Kumar, M., Longshaw, S. M., Ramos Fernandez, E., & Emerson, D. R. (2026). Multiscale universal interface v2.0: Lowering barriers to scalable multiphysics coupling. *Computer Physics Communications*, *329*, 110371. https://doi.org/10.1016/j.cpc.2026.110371

Moratilla, M. (2022). *Coupling-API: Open source API to connect OpenFOAM to Nektar++ for aeroacoustic simulations*. GitHub repository. https://github.com/mimove/Coupling-API

Moratilla-Vega, M. A., Angelino, M., Xia, H., & Page, G. J. (2022). An open-source coupled method for aeroacoustics modelling. *Computer Physics Communications*, *278*, 108420. https://doi.org/10.1016/j.cpc.2022.108420

Moureau, V., Domingo, P., & Vervisch, L. (2011). Design of a massively parallel CFD code for complex geometries. *Comptes Rendus Mécanique*, *339*(2), 141–148. https://doi.org/10.1016/j.crme.2010.12.001

nd-92. (2025). *cwipiFoam: Coupling between compressible OpenFOAM solvers and Nektar++ AcousticSolver*. GitHub repository. https://github.com/nd-92/cwipiFoam

Quémerais, E., Andrieu, B., Maugars, B., & Hoogveld, K. (2026). *ParaDiGM*. https://github.com/onera/paradigm.

Refloch, A., Courbet, B., Murrone, A., Villedieu, P., Laurent, C., Gilbank, P., Troyes, J., Tessé, L., Chaineray, G., Dargaud, J. B., Quémerais, E., & Vuillot, F. (2011). CEDRE Software. *Aerospace Lab*, *2*, p. 1–10. https://hal.science/hal-01182463

Rocamora, A., Labaune, J., Laux, C. O., & Tholin, F. (2025). Numerical study of plasma-assisted ignition and stabilisation of a supersonic hydrogen–air combustion in LAERTE facility by a quasi-DC gliding arc. *Plasma Sources Science and Technology*, *34*(10), 105002. https://doi.org/10.1088/1361-6595/ae00ee

Rubin, P. (2022). *Comparison of code coupling libraries for high performance multi-physics simulation* (DL Technical Reports DL-TR-2022-001). Science; Technology Facilities Council. https://doi.org/10.5286/dltr.2022001

Schneider, D., Schrader, T. P., & Uekermann, B. (2023). Data-parallel radial-basis function interpolation in preCICE. In M. Papadrakakis, B. Schrefler, & E. Oñate (Eds.), *X international conference on computational methods for coupled problems in science and engineering (COUPLED PROBLEMS 2023)*. CIMNE. https://doi.org/10.23967/c.coupled.2023.016

Schneider, D., & Uekermann, B. (2025). Efficient partition-of-unity radial-basis-function interpolation for coupled problems. *SIAM Journal on Scientific Computing*, *47*(2), B558–B582. https://doi.org/10.1137/24M1663843

DRAFT

Schonfeld, T., & Rudgyard, M. (1999). Steady and unsteady flow simulations using the hybrid flow solver AVBP. *AIAA Journal*, *37*(11), 1378–1385. https://doi.org/10.2514/2.636

Simon, A., François, L., & Massot, M. (2025). High-order multistep coupling: Convergence, stability and PDE application. *Comptes Rendus. Mécanique*, *353*, 1159–1184. https://doi.org/10.5802/crmeca.333

Tang, Y.-H., Kudo, S., Bian, X., Li, Z., & Karniadakis, G. E. (2015). Multiscale universal interface: A concurrent framework for coupling heterogeneous solvers. *Journal of Computational Physics*, *297*, 13–31. https://doi.org/10.1016/j.jcp.2015.05.004

Valero, M. (2026). *CONES: Coupling OpenFOAM with Numerical EnvironmentS*. GitHub repository. https://github.com/MiguelMValero/CONES

Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, *12*(6), 620–631. https://doi.org/10.1063/1.168744

DRAFT